

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Semana 7

Repaso: Primera mitad del curso

Contenido

1. Repaso: Recursividad, Ordenamiento, Punteros
2. Lectura y escritura de cadenas (cin, getline)
3. Funciones para el manejo de cadenas (cstring)
4. Funciones para caracteres (cctype)
5. Clase string moderna de C++ (STL)

1. Recursividad e Iteración en C++

Recursividad

- Una función se llama a sí misma para resolver un subproblema más pequeño.
- **Caso base:** condición de parada que evita la recursión infinita.
- **Caso recursivo:** llamada a si misma con un argumento reducido.
- Consume más memoria (stack) que la iteración; riesgo de stack overflow.

Iteración

- Repite un bloque de código usando **for, while, do-while**. Más eficiente en memoria.
- Todo algoritmo recursivo puede ser reescrito de forma iterativa.

Comparación

- **Recursión:** código más elegante y legible para problemas como árboles, factorial, Fibonacci, Torres de Hanoi.
- **Iteración:** preferible cuando el rendimiento y el uso de memoria son críticos.

Algoritmos de Ordenamiento y Búsqueda en C++

Algoritmos de Ordenamiento

- **Bubble Sort:** $O(n^2)$ - compara pares adyacentes. Simple pero ineficiente.
- **Selection Sort:** $O(n^2)$ - busca el mínimo y lo coloca en su lugar.
- **Insertion Sort:** $O(n^2)$ - inserta cada elemento en su posición correcta.
- **Merge Sort / Quick Sort:** O(n log n) - eficientes, basados en divide y vencerás.

Algoritmos de Búsqueda

- **Búsqueda Lineal (Sequential):** O(n) - recorre elemento por elemento. Funciona con arreglos no ordenados.
- **Búsqueda Binaria:** O(log n) - divide el arreglo a la mitad. **Requiere arreglo ordenado.**

STL en C++

- La STL provee **std::sort()** y **std::binary_search()** en **<algorithm>**. Eficientes y listas para usar.

Punteros, Arreglos y Matrices en C++

Punteros

- Variable que almacena una **direccion de memoria**. Declaracion: **int* p;**
- **&** (operador de direccion) y ***** (operador de desreferencia / indirección).
- Punteros nulos: inicializar con **nullptr** para evitar comportamiento indefinido.

Relacion Punteros - Arreglos

- El nombre de un arreglo es un puntero a su primer elemento: **arr == &arr[0]**
- Aritmetica de punteros: ***(arr+i) == arr[i]**

Matrices y Punteros Dobles

- Una matriz 2D es un arreglo de arreglos. Almacenada en memoria de forma contigua (fila mayor).
- Acceso: **mat[i][j] == *(*(mat+i)+j)**
- Semana 9: Memoria dinamica con **new / delete**. Alternativa moderna: **std::vector<int>**

2. Lectura y escritura de cadenas en C++

- **Lectura**: podemos ingresar una cadena de 2 formas:
  1. Usando **cin >>** **(lee una sola palabra)**
  2. Usando **getline()** con la clase **string**

```
string nombres;
    getline(cin, nombres);
```

  3. Usando **cin.getline()** con **C-strings**

```
char nombres[50];
    cin.getline(nombres, 50);
```

  Error común: Mezclar cin >> y getline()

```
cin >> edad;
cin.ignore(); // limpia el buffer
getline(cin, nombre);
```

- **Escritura**: Una cadena puede ser mostrada utilizando **cout <<**
  No necesitamos un bucle para mostrar una cadena. En C-strings, cout recorre la cadena hasta encontrar '\0'. En string cout usa el tamaño de la cadena almacenada internamente.

3. Principales funciones para el manejo de cadenas

Firma de la función	Descripción
`size_t strlen(const char* str)`	Devuelve la longitud de la cadena (sin contar el carácter nulo '\0').
`char* strcpy(char* dest, const char* src)`	Copia una cadena en otra (sobrescribe el contenido de dest).
`char* strcat(char* dest, const char* src)`	Añade (concatena) una cadena al final de otra.
`int strcmp(const char* s1, const char* s2)`	Compara dos cadenas completas (0 si son iguales).
`int strncmp(const char* s1, const char* s2, size_t n)`	Compara hasta n caracteres de dos cadenas.
`char* strchr(const char* str, int c)`	Busca la primera aparición de un carácter en la cadena.
`char* strstr(const char* s1, const char* s2)`	Busca una subcadena dentro de otra.
`char* strncpy(char* dest, const char* src, size_t n)`	Copia hasta n caracteres (puede no agregar '\0' si se llena).
`char* strtok(char* str, const char* delim)`	Divide una cadena en tokens usando delimitadores (modifica la cadena original).

Debemos incluir la librería: `#include <cstring>`

Para más detalles de otras funciones en la librería, visitar: https://cplusplus.com/reference/cstring/

4. Funciones para el manejo de caracteres

Firma de la función	Descripción
`int isalpha(int c)`	Verifica si el carácter es una letra (a–z, A–Z).
`int isdigit(int c)`	Verifica si es un dígito (0–9).
`int isalnum(int c)`	Verifica si es alfanumérico (letra o número).
`int isspace(int c)`	Verifica si es un espacio en blanco (espacio, tab, salto de línea).
`int islower(int c)`	Verifica si es una letra minúscula.
`int isupper(int c)`	Verifica si es una letra mayúscula.
`int tolower(int c)`	Convierte el carácter a minúscula.
`int toupper(int c)`	Convierte el carácter a mayúscula.
`int ispunct(int c)`	Verifica si es un signo de puntuación.
`int isprint(int c)`	Verifica si es un carácter imprimible.
`int iscntrl(int c)`	Verifica si es un carácter de control (como \n, \t).

Agregar la libreria de cabecera: `#include <cctype>`

Para mayor detalle de funciones en la librería visitar: https://cplusplus.com/reference/cctype/

3. Funciones para convertir cadenas

Firma de la función	Descripción
`int atoi(const char* str)`	Convierte una cadena a entero (int).
`long atol(const char* str)`	Convierte una cadena a entero largo (long).
`long long atoll(const char* str)`	Convierte una cadena a entero largo largo (long long).
`double atof(const char* str)`	Convierte una cadena a número decimal (double).
`long strtol(const char* str, char** endptr, int base)`	Convierte a long, permite detectar errores y especificar base (ej. base 10, 16).
`long long strtoll(const char* str, char** endptr, int base)`	Igual que strtol, pero para long long.
`double strtod(const char* str, char** endptr)`	Convierte a double con control de errores.
`float strtof(const char* str, char** endptr)`	Convierte a float con control de errores.
`long double strtold(const char* str, char** endptr)`	Convierte a long double.

Agregar la libreria de cabecera: `#include <cstdlib>`

Para mayor detalle de funciones en la librería visitar: https://cplusplus.com/reference/cstdlib/

5. Clase std::string - Tratamiento Moderno de Cadenas

¿Por que usar std::string?

- Manejo automatico de memoria (no hay riesgo de buffer overflow como en **char[]**).
- Operadores sobrecargados: concatenacion con **+**, comparacion con **== != < >**

Métodos esenciales de std::string

- **s.length() / s.size()** — longitud de la cadena.
- **s.substr(pos, len)** — extrae subcadena desde pos con longitud len.
- **s.find(sub)** — busca subcadena; retorna **string::npos** si no existe.
- **s.replace(pos, len, str)** — reemplaza len caracteres desde pos.
- **s.erase(pos, len)** — elimina len caracteres desde pos.
- **s.push_back(ch) / s.append(str)** — agrega al final.

Conversion y utilidades (C++11/17)

- **to_string(num)** — numero a string.
- **stoi() / stod() / stol()** — string a int/double/long.
- **s.c_str()** — obtiene el **const char*** compatible con C.

3. Principales funciones para el manejo de std::string

Firma / uso	Descripción
`s.size()` / `s.length()`	Devuelve la longitud de la cadena.
`s = t`	Copia una cadena en otra (reemplaza el contenido).
`s += t` / `s.append(t)`	Añade (concatena) otra cadena al final.
`s.compare(t)`	Compara dos cadenas (0 si son iguales).
`s.compare(0, n, t, 0, n)`	Compara hasta `n` caracteres de dos cadenas.
`s.find(c)`	Busca la primera aparición de un carácter.
`s.find(sub)`	Busca una subcadena dentro de otra.
`s.substr(pos, n)`	Extrae una subcadena desde una posición dada.
`s.copy(dest, n, pos)`	Copia `n` caracteres desde una posición a un arreglo `char[]`.
`getline(cin, s)`	Lee una línea completa (incluye espacios).
`s.c_str()`	Devuelve un C-string (`const char*`) equivalente.

Debemos incluir la clase: `#include <string>`

Para más detalles de otras funciones en la librería, visitar: https://cplusplus.com/reference/string/string/

3. Funciones para convertir std::string

Firma de la función	Descripción
`int stoi(const string& str)`	Convierte una cadena a entero (`int`).
`long stol(const string& str)`	Convierte a entero largo (`long`).
`long long stoll(const string& str)`	Convierte a entero largo largo (`long long`).
`float stof(const string& str)`	Convierte a número decimal (`float`).
`double stod(const string& str)`	Convierte a número decimal (`double`).
`long double stold(const string& str)`	Convierte a número decimal de mayor precisión.
`int atoi(const char* str)`	Convierte C-string a entero (`int`).
`double atof(const char* str)`	Convierte C-string a `double`.
`long atol(const char* str)`	Convierte C-string a `long`.

Agregar la libreria de cabecera: `#include <string>`

Para mayor detalle de funciones en la librería visitar: https://cplusplus.com/reference/string/string/